

Water Risk Researcher

A command-line tool that researches water-risk for any location and **proves that every data point is backed by its source.**

Follow this guide to install, configure, run, and read the results.

1. What it does

Give it a list of locations (a plant, a factory, a mine). For each one it researches three things — **water stress**, **incidents / conflicts**, and **regulations** — gathering findings from multiple web sources. Then, for every finding, it independently re-downloads the source and checks that the quoted text is really there. The result is a clean report where nothing is taken on faith.

2. How it works — three independent checks

An AI can invent a convincing statistic, a real-looking link, and a fake quote all at once. So the tool never trusts it. Every data point passes up to three separate checks, each answering a different question:

① Does the quote exist?

The source page is re-downloaded and the quoted excerpt is matched against it — **deterministically, no AI**. Shown as **Validation**.

② Does it back the claim?

A separate, closed-book AI judge checks whether the quote actually supports the data claim: **YES** / **PARTIAL** / **NO**.

③ Is the source solid?

An optional pass rates each source's authority from 0 to 5 (— **relevance**). A real quote can still come from a weak source.

Anything that can't be proven is shown as a clear failure — never hidden.

3. Requirements

- **Python 3.9 or newer.**
- **An API key for the LLM** (Claude Sonnet 5, via the Anthropic API). This is a developer key from console.anthropic.com, separate from any chat subscription. A full run costs a few US cents.
- A terminal (Terminal on macOS/Linux, PowerShell on Windows).

4. Installation

- 1 **Get the code.** Clone the repository and enter the folder:

```
git clone https://github.com/jnicovillegas/water-risk-researcher.git
cd water-risk-researcher
```

- 2 **Create an isolated environment and install.**

```
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -e .
```

5. Configure your API key

Copy the example file and paste your key into it:

```
cp .env.example .env
# then open .env and set:
ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Safe by design: the `.env` file is git-ignored, so your key never gets committed or shared. No spaces around the `=`, no quotes.

6. Running the tool

Activate the environment once per terminal session (`source .venv/bin/activate`), then run `waterrisk` .

There are three ways to tell it what to research:

Locations directly on the command line

```
waterrisk "Plant in Mexicali, Mexico" "Mine in Antofagasta, Chile"
```

From a file (best for many locations)

```
waterrisk --input locations.example.json
```

The file is a simple JSON list, e.g. `["Plant in Mexicali, Mexico", "..."]`

A full run — several formats, all checks on

```
waterrisk --input locations.example.json --format md,csv,pdf --relevance
```

While it runs you'll see live progress — each search, each source being verified — so you always know it's working. The report prints to the screen and is saved to a file.

7. Options reference

Option	What it does
<code>-i, --input FILE</code>	Read locations from a JSON file (a list of strings).
<code>-o, --output PATH</code>	Base path for the reports (default <code>out/report</code>).
<code>--format LIST</code>	Output formats: <code>md</code> , <code>csv</code> , <code>json</code> , <code>pdf</code> (default <code>md</code>). E.g. <code>--format md,csv,pdf</code> .
<code>--no-support</code>	Skip the claim-support check — seal ② (on by default).
<code>--relevance</code>	Add the source-relevance rating — seal ③ (off by default).
<code>--no-cache</code>	Force fresh searches instead of reusing saved results.
<code>--model ID</code>	Use a different model id.
<code>--quiet</code>	Don't print the report to the screen (only save it).
<code>--version</code> / <code>--help</code>	Show the version / all options.

About the cache: repeating a location returns instantly from memory. Add `--no-cache` to force a real, fresh search.

8. Reading the report

Each finding looks like this — here's what every line means:

Data: Mexicali faces extremely high baseline water stress.
↳ the factual claim.

Source: <https://www.wri.org/>... — WRI Aqueduct
↳ where it came from.

Excerpt: "...Mexicali municipality faces extremely high baseline water stress..."
↳ the exact quote from the page.

Validation:  **MATCH FOUND (exact)**
↳ check ①: the quote really exists on the page. On a fuzzy match, a *"matched on page"* line shows the exact text it aligned to.

Claim support:  **PARTIAL — the excerpt doesn't mention the score**
↳ check ②: does the quote back the claim? Names what's missing.

Source relevance: 5/5 — authoritative source
↳ check ③ (only with `--relevance`).

The summary at the top

Metric	Meaning
Verified data points	How many quotes were proven to exist on their page.
Fully supported by source	Of those, how many fully back their claim (verdict YES).
Multi-source coverage	How many dimensions have ≥ 2 <i>different</i> sources (by website).

Failures are information, not errors. A  **FAILED VALIDATION** or a **NO** means the tool did its job — it caught something that couldn't be proven, instead of passing it off as true.

9. Troubleshooting & known limits

You see...	What it means / what to do
<code>ANTHROPIC_API_KEY</code> is not set	Create the <code>.env</code> file with your key (step 5).
<code>source blocked by bot protection</code>	The site refuses automated visits. The data may still be real; it just can't be auto-verified.
<code>source unreachable</code>	The page was down, timed out, or uses an obsolete security protocol.
<code>excerpt not found</code>	The quote isn't on the live page (the model may have paraphrased). Correctly flagged.
A model id error (404)	Set a valid model with <code>--model <id></code> .

Known limits: pages built entirely with JavaScript and scanned (image-only) PDFs can't be read automatically; text-based PDFs and normal web pages are fully supported.